# Machine Learning and Agentic AI for Macroeconomic Research
# 4: Solving Macro Models via Deep Learning & RL

Zhigang Feng

Spring 2026

## Lecture Agenda

1. **Motivation: Why Deep Learning for Macro Models?** — the curse of dimensionality, function approximation, the ML opportunity
2. **The Running Example: Optimal Growth Model** — Bellman equation, VFI, analytical benchmark
3. **Deep Learning Approach** — parameterize value/policy functions with neural networks
4. **Actor-Critic Algorithm for the Growth Model** — two-network architecture, gradient-based updates
5. **Euler Equation Method + Supervised Learning** — Euler residual as loss function, training a policy network
6. **Structured Neural Networks for Equilibrium** — ICNN, monotonic networks, embedding economic theory
7. **Accuracy, Validation, and Lab Preview** — benchmarking against analytical solutions

# Motivation: Why Deep Learning for Macro Models?

# The Computational Frontier in Macro

- Quantitative macroeconomics solves dynamic optimization problems:

$$V(k) = \max_{k'} \Big\{ u\big(f(k) - k'\big) + \beta\, V(k') \Big\}$$

- **Standard toolkit**: Value Function Iteration (VFI), Euler equation methods, perturbation
- These methods have served the profession well for decades
- But they face fundamental limitations as models grow richer:
  - More state variables (capital, bonds, housing, human capital...)
  - Heterogeneous agents (wealth distributions, idiosyncratic risk)
  - Aggregate uncertainty (business cycles in HA models)
- **This lecture**: How deep learning and reinforcement learning offer a fundamentally different approach

# The Naive Approach: Grid-Based Methods

- **Value Function Iteration on a grid**:
  1. Discretize the state space: $\{k_1, k_2, \ldots, k_N\}$
  2. For each grid point, solve $\max_{k'}\{u(f(k_i) - k') + \beta V(k')\}$
  3. Iterate until convergence

- **This works well** when:
  - The state space is low-dimensional (1–3 variables)
  - The functions are smooth
  - We can afford to evaluate $V$ at every grid point

- Alternatives like Chebyshev polynomials and finite elements improve efficiency but share the same fundamental scaling problem

# Why Grids Break: The Curse of Dimensionality

- With $N$ grid points per dimension and $d$ state variables:

$$\text{Total grid points} = N^d$$

- **Example**: $N = 100$ points per dimension
  - $d = 1$: $100$ points (trivial)
  - $d = 2$: $10{,}000$ points (manageable)
  - $d = 5$: $10^{10}$ points (infeasible)
  - $d = 10$: $10^{20}$ points (absurd)

- **Real macro models** often have $d \geq 5$:
  - Krusell-Smith (1998): individual capital $k$ + aggregate capital $K$ + shocks
  - Life-cycle models: age + assets + earnings + health + ...
  - HA models with aggregate risk: the *entire wealth distribution* is a state

- Grid-based methods simply **cannot** scale to these problems

## Beyond Grids: Other Classical Limitations

- **Perturbation methods** (linearization around steady state):
  - Fast and accurate locally
  - But may lose accuracy far from steady state
  - Struggle with non-linearities, occasionally binding constraints, regime switches
- **Projection methods** (Chebyshev, finite elements):
  - Globally accurate for smooth problems
  - Require structured grids or node sets
  - Still suffer from the curse of dimensionality
- **Kinks and non-convexities**:
  - Borrowing constraints, irreversible investment, discrete choices
  - First-order conditions fail or require complex root-finding
- We need function approximators that scale gracefully with dimension and handle irregular structures

# The Deep Learning Opportunity

- **Neural networks** are universal function approximators:
  - Can represent any continuous function on a compact set (Cybenko 1989, Hornik et al. 1989)
  - Parameter count grows *polynomially* with dimension, not exponentially
- **Automatic differentiation** provides gradients for free:
  - No need to derive and code analytical derivatives
  - Euler equations, FOCs, Jacobians — all computed automatically
- **GPU acceleration**:
  - Matrix operations (forward/backward pass) are massively parallelizable
  - Simulate thousands of economic agents simultaneously
- **The key insight**: Recast the macro model as a *learning problem* where the decision rule (policy) or value function is the object to be learned

# The Inspiration: AlphaGo and Economic Models

- **AlphaGo** (DeepMind, 2016) solved a problem with $\sim 10^{170}$ possible board states
- It did *not* enumerate the state space or solve the full Bellman equation on a grid
- **It used two neural networks**:
  - **Policy Net (Actor)**: "Intuition" — which move to play
  - **Value Net (Critic)**: "Judgment" — who is winning
- **Lesson for economics**: When the state space is too vast to enumerate (like the wealth distribution in a HA model), we can use the same two-network approach
- **This lecture**: Apply this actor-critic idea to the optimal growth model as a proof of concept, then discuss how it scales to richer models

# Reframing Macro as a Machine Learning Problem

- The optimal growth model can be recast as a **learning problem**:
  - The decision rule $g(k)$ is approximated by a neural network
  - The first-order conditions provide a natural **loss function**
  - Modern optimization (Adam, SGD) and automatic differentiation drive training

- **Two complementary approaches**:
  1. **Deep Value Function Iteration**: Train networks to satisfy the Bellman equation (reinforcement learning flavor)
  2. **Euler Equation Minimization**: Train a network to satisfy the FOCs (supervised learning flavor)

- Both use the same building blocks:
  - Neural network function approximation
  - Stochastic gradient descent
  - Monte Carlo sampling of state space

# Running Example: The Optimal Growth Model

# Optimal Growth Model: Setup

- **Objective**: Maximize discounted lifetime utility:

$$\max_{\{k_{t+1}\}} \sum_{t=0}^{\infty} \beta^t u\Big(f(k_t) - k_{t+1}\Big)$$

- **Production function**: $f(k_t) = k_t^{\alpha}$
- **Resource constraint**: Consumption $c_t = f(k_t) - k_{t+1}$
- **Depreciation**: $\delta = 1$ (full depreciation) in our benchmark; adjustable for general cases
- **Steady-state capital**:

$$k_{ss} = \left(\frac{1}{\alpha\beta}\right)^{\frac{1}{\alpha-1}} = (\alpha\beta)^{\frac{1}{1-\alpha}}$$

- **Why this model?** Simple enough for an analytical solution (ground truth), yet contains all the essential structure of richer models

# Bellman Equation and VFI

- **Recursive formulation** (Bellman equation):

$$V(k) = \max_{k'} \Big\{ u\big(f(k) - k'\big) + \beta\, V(k') \Big\}$$

- **Value Function Iteration**:
  1. Start with initial guess $V^0(k)$
  2. Update: $V^{n+1}(k) = \max_{k'} \{u(f(k) - k') + \beta\, V^n(k')\}$
  3. Repeat until $\|V^{n+1} - V^n\| <$ tolerance

- **Contraction Mapping Theorem**: Under standard assumptions, the Bellman operator $T$ is a contraction with modulus $\beta$
  - Guarantees convergence to unique fixed point $V^*$
  - This is the theoretical foundation we build on

## Stochastic Extension

- Add a productivity shock $z$:

$$v(y) = \max_{0 \le c \le y} \Big\{ u(c) + \beta \, \mathbb{E}\big[v(y')\big] \Big\}$$

- Transition: $y' = f(y - c) \cdot z$, where $z$ is i.i.d. log-normal
- **Analytical solution** for log utility $u(c) = \ln(c)$ and Cobb-Douglas $f(k) = k^{\alpha}$:

$$V^*(y) = A + B \ln y \quad \text{(value function)}$$
$$c^*(y) = (1 - \alpha\beta)\, y \quad \text{(optimal policy: consume fixed fraction)}$$

- This closed-form solution serves as our **ground truth** for validating all neural network methods
- Lab 6A builds this benchmark via classical VFI with interpolation

# VFI with Interpolation: The Classical Algorithm

- Since $y$ is continuous, we use interpolation on a grid:

  1. **Initialize**: Grid $\{y_1, \ldots, y_N\}$, initial guess $v^0$
  2. **Interpolate**: Create function $\hat{v}(y)$ connecting $(y_i, v_i^0)$ pairs
  3. **Bellman update**: For each $y_i$:

$$v_i^{\text{new}} = \max_c \left\{ \ln(c) + \beta \sum_j \hat{v}\big(f(y_i - c)\, z_j\big)\, \phi(z_j) \right\}$$

  4. **Check convergence**: If $\|v^{\text{new}} - v^0\| < \text{tol}$, stop; else repeat

- This is the **benchmark** algorithm implemented in Lab 6A
- Works beautifully for 1D... but the grid makes it impractical for higher dimensions

# Deep Learning Approach: NNs for Value and Policy Functions

# The Core Idea: Replace Grids with Neural Networks

- **Instead of** storing $V(k)$ on a grid, **approximate** it with a neural network:

$$V(k) \approx \Gamma_{\gamma_v}(k), \qquad g(k) \approx \Gamma_{\gamma_g}(k)$$

  where $\gamma_v, \gamma_g$ are network parameters (weights and biases)

- **The analogy to VFI**:
  1. Generate states $k \sim d(k)$ from some distribution (replaces grid)
  2. Compute approximate target values or gradients for training
  3. Update $\gamma_v$ or $\gamma_g$ using gradient-based optimization

- **Why this helps**: Neural networks handle high-dimensional $k$ naturally
  - No grid needed — states are *sampled*
  - The network generalizes across the state space
  - Parameters grow polynomially, not exponentially, with dimension

# Embedding Equilibrium Conditions in Neural Networks

- Neural network approximations can incorporate **equilibrium conditions** directly:
  - **Feasibility constraints**: Ensure $g(k) \in \mathcal{G}(k)$
    - Example: $0 \leq c \leq f(k)$ (can't consume more than output)
  - **Market clearing**: $\sum_i c_i = Y$ or asset market clearing
  - **Budget constraints**: Household or government budget balance

- **Implementation strategies**:
  1. **Output transformations**: Sigmoid $\rightarrow$ bound outputs within feasible range
  2. **Penalty methods**: Add constraint violation terms to the loss
  3. **Lagrangian approaches**: Dual variables updated during training
  4. **Architectural design**: Structure the network so outputs automatically satisfy identities

- Building in economic structure improves convergence and ensures economically meaningful solutions

# Value Function Update (One-Step Look-Ahead)

- Fix a policy $g^{(n-1)}(k) = \Gamma_{\gamma_g}^{(n-1)}(k)$
- The one-step approximation for the value is:

$$\hat{V}(k) = u\big(k, g^{(n-1)}(k)\big) + \beta\, \Gamma_{\gamma_v}^{(n-1)}\big(f(k, g^{(n-1)}(k))\big)$$

- We then solve a **least squares** problem:

$$\min_{\gamma_v^{(n)}} \frac{1}{N_v} \sum_{i=1}^{N_v} \Big(\Gamma_{\gamma_v}^{(n)}(k_i) - \hat{V}(k_i)\Big)^2$$

- **Key**: States $\{k_i\}$ are *sampled* from distribution $d(k)$, not placed on a grid
- This is **supervised learning**: the "label" for each $k_i$ is the Bellman target $\hat{V}(k_i)$

# The Role of Value Function in Policy Evaluation

- Note that $\Gamma_{\gamma_v}^{(n-1)}(k)$ is **fixed** when computing $\hat{V}(k)$
- **Key insight**: The value network serves as an *evaluator* of the current policy:
  - If $\Gamma_{\gamma_v}^{(n-1)} \approx V^*$ and $g^{(n-1)} \approx g^*$, then $\hat{V}(k) \approx V^*(k)$
- **On the expectation operator**: $\mathbb{E}_{k\sim d(k)}[\cdot]$ averages over the *state space*, not over stochastic uncertainty
  - Even in deterministic models, we average over sampled states
  - **Rationale**: If a candidate policy performs well *on average* across all sampled states, it must be a good approximation globally
  - Poor average performance signals the policy is inadequate somewhere
- This "average fitness" criterion ensures robust approximation across the entire domain

# Policy Function Update

- Given the updated value network $\Gamma_{\gamma_v}^{(n)}$, improve the policy by solving:

$$\max_{\gamma_g^{(n)}} \frac{1}{N_g} \sum_{i=1}^{N_g} \Big[ u\big(k_i, \Gamma_{\gamma_g}^{(n)}(k_i)\big) + \beta\, \Gamma_{\gamma_v}^{(n)}\big(f(k_i, \Gamma_{\gamma_g}^{(n)}(k_i))\big) \Big]$$

- This is **gradient ascent** on the neural network parameters:

$$\gamma_g^{(n)} = \gamma_g^{(n-1)} + \alpha \cdot \nabla_{\gamma_g} J(\gamma_g)\Big|_{\gamma_g = \gamma_g^{(n-1)}}$$

- **Two roles of the gradient**:
  1. **Direction**: $\nabla_{\gamma_g} J$ tells us *which direction* in parameter space improves the objective
  2. **Magnitude**: Combined with learning rate $\alpha$, determines *how far* we move

- In practice, optimizers like Adam adaptively adjust step sizes for faster and more stable convergence

## Choosing the Sampling Distribution $d(k)$

- The distribution $d(k)$ determines *where* we evaluate the approximation
- **Common choices**:
  - **Uniform**: Simple, but wastes capacity on unlikely states
  - **Stationary distribution**: Focuses on states the economy actually visits
  - **Importance sampling**: Targets high-uncertainty or high-value regions
- **Stationary distribution approach**:
  1. Simulate the economy under current policy for many periods
  2. Wait until the state converges to its ergodic distribution
  3. Draw training points from that distribution
- **Benefits**: Sample efficiency (focus on what matters), faster convergence, no need for a handcrafted grid
- **Limitation**: Rare but important events (tail risks) may be under-represented

# Why Not Chebyshev Here?

- Chebyshev and finite-element methods must approximate over the *entire* domain
- As dimensionality grows, covering the entire domain becomes infeasible
- **Stationarity-based sampling** zooms in on the *likely* region of interest
- **Possible concerns and remedies**:
  - **Rare but important events**: Combine stationary sampling with targeted sampling of rare states (importance sampling)
  - **Multiple steady states**: A single stationary distribution might miss transient states
  - **Out-of-distribution robustness**: Conduct stress tests separately
- The combination of **Monte Carlo sampling** (generates the right data in relevant states) and **neural networks** (fits irregular, mesh-free data) is the key synergy

# Bias-Variance Tradeoff: One-Step vs. Multi-Step

- **One-step** ($T_{\text{sim}} = 1$):

$$\hat{V}(k_i) = u\big(k_i, g(k_i)\big) + \beta\, \Gamma_{\gamma_v}(k_{i,1})$$

  + Easy to implement, low variance
  − High bias if $V(\cdot)$ approximation is poor

- $T_{\text{sim}}$**-step return**:

$$\hat{V}(k_i) = \sum_{t=0}^{T_{\text{sim}}-1} \beta^t\, u(k_{i,t}, g(k_{i,t})) + \beta^{T_{\text{sim}}}\, \Gamma_{\gamma_v}(k_{i,T_{\text{sim}}})$$

  + Lower bias: uses actual simulated rewards over multiple steps
  − Higher variance: simulated paths can vary significantly

- **In practice**: Choose $T_{\text{sim}}$ to balance bias vs. variance; Lab 6B uses $T_{\text{sim}} = 30$–$50$

# Actor-Critic Algorithm for the Growth Model

# Why Actor-Critic? The Optimization Bottleneck

*Preview: we develop the full RL theory in Lecture 5; here we apply the actor-critic idea to the growth model.*

- **Classical VFI** requires solving $\max_{k'} Q(k, k')$ at every state
- When $Q$ is represented by a neural network (non-convex), this inner maximization is expensive and unreliable
- **The actor-critic solution**: Use *two* networks to avoid the inner max:
  1. **The Actor (Policy Net $\pi$)** — "The Doer"
     - Input: state $k \to$ Output: action $k'$ directly
     - *Avoids* the expensive maximization step
  2. **The Critic (Value Net $V$)** — "The Judge"
     - Input: state $k \to$ Output: estimated value $V(k)$
     - Minimizes the **Bellman residual** (prediction error)
- The actor proposes actions; the critic evaluates them; both improve iteratively

# Mapping Macroeconomics to Reinforcement Learning

To apply RL, we translate economic models into the standard tuple $(S, A, P, R, \gamma)$:

| RL Concept | Economic Equivalent | Note |
|---|---|---|
| **Agent** | Household / Firm | The optimizer |
| **Environment** | Constraints & Prices | The rules of the game |
| **State ($S$)** | $(k, z, \Delta)$ | Wealth, shocks, **distribution** |
| **Action ($A$)** | $c, i, l$ | Consumption, investment |
| **Transition ($P$)** | $k' = (1-\delta)k + i$ | **Micro**: Known perfectly |
| | $\Delta' = H(\Delta)$ | **Macro**: Often unknown/intractable |
| **Reward ($R$)** | $u(c)$ | Period utility |
| **Discount ($\gamma$)** | $\beta$ | Time preference |

**The synergy**: RL handles the complexity of the macro transition $\Delta' = H(\Delta)$ by simply *simulating* it, without needing to derive the law of motion explicitly

# The Actor-Critic Algorithm: Overview

1. **Initialize**: Policy network $\Gamma_{\gamma_g}^{(0)}$, Value network $\Gamma_{\gamma_v}^{(0)}$
2. **Policy Evaluation** (update the Critic):
   - Generate training data $\{k_i, \hat{V}(k_i)\}$ using $T_{\text{sim}}$-step Monte Carlo simulation
   - Solve: $\min_{\gamma_v^{(n)}} \sum_i [\Gamma_{\gamma_v}^{(n)}(k_i) - \hat{V}(k_i)]^2$
3. **Policy Improvement** (update the Actor):
   - Generate states $\{k_i\}$ (e.g., from stationary distribution)
   - Solve: $\max_{\gamma_g^{(n)}} \sum_i [u(k_i, \Gamma_{\gamma_g}^{(n)}(k_i)) + \beta\, \Gamma_{\gamma_v}^{(n)}(f(k_i, \Gamma_{\gamma_g}^{(n)}(k_i)))]$
4. **Check convergence** and repeat until $\Gamma_{\gamma_g}$ and $\Gamma_{\gamma_v}$ stabilize

# Addressing the Infinite Horizon

**The puzzle**: Macro models maximize over an infinite horizon ($\sum_{t=0}^{\infty} \beta^t u_t$), but Monte Carlo simulations are always finite.

**The solution: Bootstrapping**

- We don't need to simulate to infinity
- We use the **estimated value function** (Critic) as a proxy for the infinite future:

$$\text{Total Value} \approx \underbrace{\sum_{t=0}^{T-1} \beta^t u_t}_{\text{Simulated rewards}} + \beta^T \underbrace{\hat{V}(s_T)}_{\text{Learned proxy for } \sum_{t=T}^{\infty}}$$

- This allows learning infinite-horizon policies using only short simulation rollouts
- The Critic improves over time, making the bootstrap estimate more accurate

# Monte Carlo and Uncertainty

- When the model includes **stochastic shocks**, the future state evolves randomly
- Enumerating all possible future states or integrating over a high-dimensional shock space quickly becomes infeasible (another curse of dimensionality)
- **Monte Carlo simulation** circumvents this:
  - Generate a large sample of realized state-and-shock paths
  - Compute empirical averages to approximate expectations
  - No need to explicitly deal with the full distribution
- **In practice**:
  - Choose $T_{\text{sim}}$ to balance bias vs. variance
  - Use parallel or GPU-based Monte Carlo for large simulation demands
  - Combine with importance sampling if rare shocks are of interest

# Advantage I: Flexible Function Approximation

- **Old way — Structured grids**:
  - Requires tensor product grids or sparse grids (Smolyak)
  - Adding one state variable *multiplies* the grid size
- **New way — Neural networks**:
  - Universal approximators: can represent any continuous function
  - Parameter count grows linearly/polynomially with dimension, *not* exponentially
  - Can handle high-dimensional states (entire shock histories, fine-grained distributions) that break grid-based methods
- **Result**: Models that were previously intractable become solvable

# Advantage II: The "Mesh-Free" Synergy

Why is the combination of **Monte Carlo** and **deep learning** so powerful?

- **Monte Carlo simulation** produces a "cloud" of points concentrated in the **ergodic set** (economically relevant states)
  - Traditional interpolators (Chebyshev, splines) *fail* on such irregular, clustered data (Runge's phenomenon)
- **Neural networks are "mesh-free"**:
  - They do not require a grid
  - They simply minimize error on *whatever* points you give them
- **Conclusion**: MC generates the *right data* (relevant states), and DL is the *right tool* to fit that irregular data

# Advantage III: Hardware Acceleration

- **Parallelism**:
  - Simulate thousands of economic agents in parallel on a GPU
  - Neural network training (matrix multiplication) is highly optimized for GPUs/TPUs
- **Comparison**:
  - Traditional VFI is often serial or hard to parallelize efficiently
  - Deep RL scales up naturally: more agents, more shocks $\rightarrow$ just add more compute
- **Practical implication**: Models that took hours or days can now be solved in minutes
  - Enables rapid iteration on model specification
  - Makes calibration and estimation feasible for complex models

# Euler Equation Method + Supervised Learning

# Context: Two Approaches to the Same Problem

- We have seen the **actor-critic** approach: train two networks to satisfy the Bellman equation
- Now we consider an alternative: **Euler equation minimization**
  - Derive the first-order conditions (Euler equations)
  - Train a *single* policy network to satisfy them
- **Comparison**:
  - Actor-Critic (Bellman): Two networks, no need to derive FOCs, RL flavor
  - Euler Equation: One network, requires deriving FOCs, supervised learning flavor
- Both are valid; which is better depends on the model:
  - Euler equation method is very precise when FOCs are available
  - Actor-Critic is more general (handles kinks, discrete choices)

# The Euler Equation: Optimality Condition

- Capital accumulation: $k' = g(k)$, consumption: $c = f(k) + (1-\delta)k - g(k)$
- For CRRA utility $u(c) = \frac{c^{1-\sigma}}{1-\sigma}$ and Cobb-Douglas production, the equilibrium satisfies:

$$u'(c) = \beta\, u'(c')\left[\alpha\, g(k)^{\alpha-1} + 1 - \delta\right] \tag{EE}$$

- **Interpretation**: The marginal cost of saving today equals the discounted marginal benefit tomorrow
- **Goal**: Learn a policy $g$ that makes the **Euler residual** vanish:

$$\mathcal{R}(k; g) = u'(c) - \beta\, u'(c')\big[\alpha\, g(k)^{\alpha-1} + 1 - \delta\big]$$

for all economically relevant $k$

# Euler Equation: Necessary but Not Sufficient

- **Important caveat**: The Euler equation is a *necessary* condition for optimality, but **not sufficient** by itself
- For a complete characterization, we also need:
  - **Transversality condition**: $\lim_{t\to\infty} \beta^t u'(c_t)\, k_{t+1} = 0$
  - **Feasibility constraints**: $c \geq 0$, $k' \geq 0$
- **Handling inequality constraints**:
  - Kuhn-Tucker complementary slackness conditions arise
  - Can be converted to equalities via Fischer-Burmeister functions or penalty methods
  - Or handled via architectural constraints (sigmoid output)
- For now, we focus on **interior solutions** where the Euler equation holds with equality throughout

## From Euler Residual to Loss Function

- Approximate the policy by a neural network: $g(k) \approx \Gamma_{\gamma_g}(k)$
- Sample $k_i \sim d(k)$ (e.g., stationary distribution or uniform)
- Define the **mean-squared Euler residual** loss:

$$\mathcal{L}_g(\gamma_g) = \frac{1}{N} \sum_{i=1}^{N} \left[\mathcal{R}(k_i; \Gamma_{\gamma_g})\right]^2$$

- **This is supervised learning**: the "label" for each state $k_i$ is zero (the Euler equation should hold exactly)
- Optimal policy parameters satisfy: $\nabla_{\gamma_g} \mathcal{L}_g(\gamma_g^*) = 0$
- All derivatives computed automatically via `autograd`

# The Mean Value Intuition

- We minimize the *average* Euler residual across sampled states:

$$\mathcal{L}_g(\gamma_g) = \mathbb{E}_{k\sim d(k)}\left[\mathcal{R}(k;\Gamma_{\gamma_g})^2\right] \approx \frac{1}{N}\sum_{i=1}^{N}\mathcal{R}(k_i;\Gamma_{\gamma_g})^2$$

- **Intuition**:
  - If a candidate policy satisfies the Euler equation *on average* across all sampled states, it should be a good approximation to the true equilibrium policy
  - If the average residual is large, the policy must be violating optimality somewhere

- This "average fitness" criterion:
  - Ensures global accuracy across the domain
  - Is computationally tractable via Monte Carlo sampling
  - Naturally integrates with gradient-based NN training

# Neural Network for the Policy Function

- Two-hidden-layer MLP (64–64–1) with ReLU and Sigmoid:

$$\boxed{\Gamma_{\gamma_g}(k) = \sigma_2\Big(W_2 \cdot \sigma_1(W_1 k + b_1) + b_2\Big)}$$

- Hidden activation $\sigma_1 = \texttt{ReLU}$; output activation $\sigma_2 = \texttt{Sigmoid}$
- Sigmoid output is scaled to $[0, f(k) + (1-\delta)k - \varepsilon]$ to ensure feasibility:

$$0 \leq \Gamma_{\gamma_g}(k) \leq f(k) + (1-\delta)k$$

- All derivatives $\partial \mathcal{L}_g / \partial \gamma_g$ are computed automatically by `autograd`
- No need to derive or code analytical Jacobians

# Minimising the Euler Residual: Training Loop

- Initialize $\gamma_g^{(0)}$; choose learning rate $\alpha_g$
- **Repeat until convergence**:
  1. Draw a mini-batch $\{k_i\}_{i=1}^{B}$
  2. Compute loss $\mathcal{L}_g(\gamma_g)$ and its gradient via back-propagation
  3. Gradient step:
$$\gamma_g^{(j+1)} = \gamma_g^{(j)} - \alpha_g \, \nabla_{\gamma_g} \mathcal{L}_g(\gamma_g^{(j)})$$
  4. Optionally resample $k_i$ from the same distribution $d(k)$
- **Pros of Euler method**:
  - Very precise policy functions; theoretically grounded
  - Only one network to train (simpler than actor-critic)
- **Cons**: Does not yield the value function $V$; requires deriving FOCs

# PyTorch Implementation

# Implementation Overview: Two Approaches

- We implement both approaches in PyTorch:
- **Approach 1 — Euler Equation (Lab 6B, Part 1)**:
  - Single `PolicyNetwork` with constrained output
  - Loss: Mean squared Euler error
  - Result: Policy function $g(k)$
- **Approach 2 — Deep Value Function Iteration (Lab 6B, Part 2)**:
  - Two networks: `PolicyNet` + `ValueNet`
  - $T$-step lookahead with soft constraints
  - Result: Both policy $g(k)$ and value function $V(k)$
- Both validated against the analytical solution from Lab 6A

# Network Architecture: PolicyNet and ValueNet

```python
class ValueNet(nn.Module):
    def __init__(self, nh=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, nh), nn.ReLU(),
            nn.Linear(nh, nh), nn.ReLU(),
            nn.Linear(nh, 1))
    def forward(self, k): return self.net(k)

class PolicyNet(nn.Module):
    def __init__(self, alpha, eps, nh=64):
        super().__init__()
        self.alpha, self.eps = alpha, eps
        self.net = nn.Sequential(
            nn.Linear(1, nh), nn.ReLU(),
            nn.Linear(nh, nh), nn.ReLU(),
            nn.Linear(nh, 1), nn.Sigmoid())  # Output in [0, 1]
    def forward(self, k):
        max_kp = torch.clamp(k**self.alpha - self.eps, 1e-9)
        return self.net(k) * max_kp  # Scale to [0, f(k)-eps]
```

- `Sigmoid` ensures $g(k) \in [0, f(k) - \varepsilon]$: feasibility by construction
- `ValueNet`: unconstrained output (value can be any real number)

# Policy Evaluation: Bellman MSE

```
def value_update_step(self):
    self.value_optimizer.zero_grad()
    # Sample batch of capital states
    k_batch = torch.rand(self.n_batch, 1, device=self.device) \
              * (self.k_max - self.k_min) + self.k_min

    v_current = self.value_net(k_batch)           # V(k)
    kp_batch  = self.policy_net(k_batch)           # k' = g(k)
    u_batch   = self.utility(k_batch, kp_batch)    # u(c)
    v_next    = self.value_net(kp_batch).detach()   # V(k') -- detach!
    bellman_rhs = u_batch + self.beta * v_next      # u + beta*V(k')

    loss = nn.functional.mse_loss(v_current, bellman_rhs)
    loss.backward()
    self.value_optimizer.step()
    return loss.item()
```

- Minimizes $\mathbb{E}[(V(k) - (u + \beta V(k')))^2]$
- `detach()`: prevents gradients from flowing through $V(k')$ — treats the target as fixed

# Policy Improvement: Gradient Ascent

```python
def policy_step(self):
    self.policy_optimizer.zero_grad()
    # Sample batch of capital states
    k_batch = torch.rand(self.n_batch, 1, device=self.device) \
              * (self.k_max - self.k_min) + self.k_min

    kp_batch = self.policy_net(k_batch)              # k' = g(k)
    u_batch  = self.utility(k_batch, kp_batch)       # u(c)
    v_next   = self.value_net(kp_batch)               # V(k')

    objective = torch.mean(u_batch + self.beta * v_next)

    loss = -objective   # Maximize objective = Minimize -objective
    loss.backward()
    self.policy_optimizer.step()
    return objective.item()
```

- Maximizes $\mathbb{E}[u(k, g(k)) + \beta\, V(g(k))]$
- Gradients flow through **both** $u$ and $V$ into the policy network parameters

# The Training Loop: Alternating Updates

```python
def train(self):
    print(f"Training started on device={self.device}")
    pbar = tqdm(range(self.n_epoch), desc="Outer Loop")

    for _ in pbar:
        # Phase 1: Update Value Net (several inner steps)
        for _ in range(self.n_inner_value):
            v_loss = self.value_update_step()
        self.value_losses.append(v_loss)

        # Phase 2: Update Policy Net (several inner steps)
        for _ in range(self.n_inner_policy):
            policy_obj = self.policy_update_step()
        self.policy_objectives.append(policy_obj)

        pbar.set_postfix({
            "ValueLoss": f"{v_loss:.4e}",
            "PolicyObj": f"{policy_obj:.4e}"
        })
```

- Alternates between updating the Critic and the Actor
- Multiple inner steps per outer iteration for stability

# Deep Value Function Iteration (DVFI): Key Innovations

Lab 6B, Part 2 introduces three innovations beyond the basic actor-critic:

1. **Unified Objective ($J$)**: $T$-step discounted sum plus terminal value:

$$J(y_0) = \sum_{t=0}^{T-1} \beta^t u(c_t) + \beta^T V(y_T)$$

2. $T$**-Step Lookahead**: Simulate the economy for $T = 30$–$50$ periods
   - Gradients "see" long-term consequences of policy choices
   - Reduces bias compared to one-step bootstrapping

3. **Soft Constraints** (penalty method):
   - If $c > 0$: $u = \ln(c)$ (standard)
   - If $c \leq 0$: $u =$ Penalty $\times (c - \varepsilon)$ (pushes back to feasibility)
   - Avoids hard clamping that kills gradients

# DVFI: Separated Training Phases

- **Phase A — Policy Net** (maximize $J$):
  - Sample initial states $y_0$ uniformly in $[y_{\min}, y_{\max}]$
  - Simulate $T$ periods using PolicyNet
  - Compute $J = \sum \beta^t u(c_t) + \beta^T V(y_T)$
  - Minimize $-J$ (gradient ascent on expected reward)
- **Phase B — Value Net** (fit $V$ to $J$):
  - Target: $J$ from Phase A, **detached** from policy gradients
  - Loss: $\|V(y_0) - J\|^2$ (standard regression)
- **Key detail**: `retain_graph=True` allows both phases to share the same computation graph; `detach()` in Phase B prevents policy updates through the value loss

# Structured Neural Networks for Equilibrium Models

# The Computational Challenge in Quantitative Macro

- **Nested "Russian Doll" structure**: Quantitative macro models involve *multiple* nested optimization problems
- **Typical hierarchy**:
  1. **Outer Loop**: Market clearing / calibration (solve for prices $p^*$)
  2. **Middle Loop**: Value function iteration (solve Bellman equations)
  3. **Inner Loop**: Agent optimization (choose $c, \ell, k'$ given state and prices)
- **Problem with unstructured nets**: Approximating value functions or costs with generic FFNNs turns the *inner* problem into a non-convex optimization
  - Multiple local optima and unstable outer iterations
  - Economic misspecification (e.g., upward-sloping demand, non-concave value functions)

# Idea: Inject Economic Structure into the Network

**Key idea**: Design architectures that *respect* economic shape restrictions *by construction*.

| **Property** | **Mathematical Gain** | **Economic Payoff** |
|---|---|---|
| **Convexity / Concavity** | Unique global optimum | Stable policy rules / VFI |
| **Monotonicity** | Invertible mappings | Well-behaved supply/demand |
| **Homogeneity** | $f(\lambda x) = \lambda f(x)$ | Consistency with balanced growth |

- This turns the neural net from a **black box** into a **glass box**: flexible enough to fit data, but disciplined by theory

# Baseline: Standard Feed-Forward Networks

**Generic FFNN architecture**:

- Composition of affine maps and nonlinearities:

$$y = f_\theta(x) = W_L\, \sigma\big(\dots \sigma(W_1 x + b_1)\big)$$

- Layer weights $W^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ are **unconstrained**

**The optimization trap**:

- Even if the *true* value function is concave, the FFNN approximation generally is not
- When agents optimize against this approximation:
  - FOCs are necessary but not sufficient
  - Numerical solvers can converge to spurious local optima
  - Comparative statics and equilibrium mappings can be erratic

- **Solution**: Constrain the network architecture to guarantee the desired shape

# Input Convex Neural Networks (ICNN)

**Goal**: Construct $f(x)$ that is convex in $x$ *by design*.

**Recursive definition**:

$$z_{l+1} = \sigma_l \left( \sum_{i=0}^{l} W_{l,i}^{(z)} z_i + W_l^{(x)} x + b_l \right)$$

**Sufficient conditions for convexity**:

1. **Non-negative recurrent weights**: all $W_{l,i}^{(z)} \geq 0$
2. **Convex, non-decreasing activations**: $\sigma_l$ is convex and non-decreasing (e.g., ReLU, Softplus)

**Result**:

- $f(x)$ is convex in $x$ as a composition of convex, non-decreasing functions
- **Crucially**: Direct input weights $W_l^{(x)}$ can be **unconstrained** (including negative)
- This allows the network to learn the linear placement and slope freely

# Why Impose Convexity in Macro?

Using ICNNs for value functions $V$ or pricing kernels delivers three benefits:

1. **Robust Value Function Iteration**
   - We solve $\max_{x'}\{u(x, x') + \beta \hat{V}(x')\}$
   - If $\hat{V}$ is concave (or $-\hat{V}$ convex via ICNN), the objective is globally concave
   - Fast, gradient-based *convex* optimization with no local-maxima issues
2. **Well-Behaved Equilibrium Prices**
   - Convex preferences $\Rightarrow$ monotone demand
   - Unique, stable market-clearing price vectors
3. **Scalability to High Dimensions**
   - ICNNs scale to high-dimensional state spaces while retaining the structure needed for Bellman contractions

# ICNN: Proof Sketch (Why It's Convex)

We prove by induction that each $z_l(x)$ is convex in $x$.

**Step 1: Base Case**

- $z_0(x) = x$ is linear, hence convex ✓

**Step 2: Inductive Step**

$$z_{l+1}(x) = \sigma\big(W_z^{(l)} z_l(x) + W_x^{(l)} x + b_l\big)$$

- By induction, $z_l(x)$ is convex
- With $W_z^{(l)} \geq 0$: $W_z^{(l)} z_l(x)$ is a non-negative sum of convex functions $\Rightarrow$ convex
- Adding linear term $W_x^{(l)} x + b_l$ preserves convexity
- Applying convex, non-decreasing $\sigma(\cdot)$ to a convex argument preserves convexity ✓

**Therefore**: All layers $z_l(x)$, and hence $f(x)$, are convex in $x$

## Do Constraints Limit Expressivity?

**The concern**: By restricting $W_z \geq 0$, do we lose the universal approximation property?

**Theorem** (Chen, Man & Amos, 2019):

*An Input Convex Neural Network can approximate any convex function over a compact domain to arbitrary accuracy.*

**Intuition**:

- A maximum of affine functions ($\max_i \{a_i^\top x + b_i\}$) is convex and can approximate any convex shape
- An ICNN with ReLU activations acts as a hierarchical max-affine approximator

**Implication for macro**:

- You are not imposing a specific parametric form (like CES or Cobb-Douglas)
- You are imposing a **shape restriction** (convexity) while retaining non-parametric flexibility

# Partial Convexity: The Macro Reality

**The issue**: Value functions $V(k, z)$ are typically:

- **Concave** in endogenous states $k$ (capital, assets)
- **Non-concave / arbitrary** in exogenous states $z$ (TFP, income shocks)

A standard ICNN would incorrectly enforce convexity on $z$ as well.

**The solution: Partial ICNN (PICNN)**

Modify the architecture to distinguish between convex inputs ($u$) and non-convex inputs ($v$):

$$z_{l+1} = \sigma\left(\underbrace{W_z^{(l)} z_l}_{\geq 0} + \underbrace{W_u^{(l)}(u \circ v)}_{\text{interaction}} + \underbrace{W_v^{(l)} v}_{\text{free}} + b_l\right)$$

**Implementation**: Pass $k$ through the "convex path" (constrained weights); pass $z$ through a standard FFNN path; combine such that convexity w.r.t. $k$ is preserved

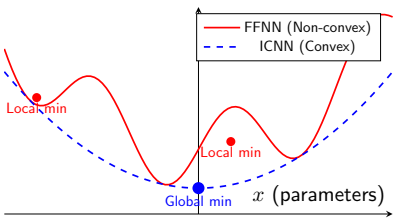
# Optimization Landscapes: FFNN vs. ICNN

**Standard FFNN**

- Rugged, non-convex landscape
- Many local minima
- Outcome depends on initialization

**ICNN**

- Smooth, convex basin
- Unique global minimum
- Optimization is predictable



# ICNN: Robust PyTorch Implementation

**Strategy**: Use reparameterization (Softplus) instead of clamping to enforce $W_z \geq 0$.

```python
class ICNN(nn.Module):
    def __init__(self, d_in, hidden_sizes):
        super().__init__()
        # Passthrough weights (Input -> Hidden) [unconstrained]
        self.W_x = nn.ParameterList([
            nn.Parameter(torch.randn(h, d_in))
            for h in hidden_sizes])
        # Recurrent weights (Hidden -> Hidden) [unconstrained params]
        # Apply Softplus() during forward pass to enforce W_z >= 0
        self.W_z_params = nn.ParameterList([
            nn.Parameter(torch.randn(h, h_prev))
            for h, h_prev in zip(hidden_sizes[1:], hidden_sizes[:-1])])
        self.biases = nn.ParameterList([
            nn.Parameter(torch.zeros(h)) for h in hidden_sizes])

    def forward(self, x):
        z = x
        for i, (Wz_p, Wx, b) in enumerate(
                zip(self.W_z_params, self.W_x, self.biases)):
            Wz = F.softplus(Wz_p)  # Enforce non-negativity
            z_next = F.linear(z, Wz) + F.linear(x, Wx) + b
            z = F.relu(z_next)
        return z
```

# Monotonic Neural Networks

**Objective**: Ensure $f(x)$ is non-decreasing:

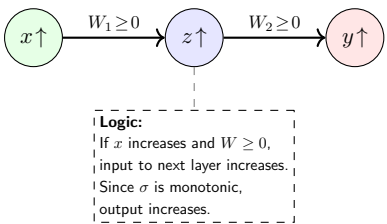
$$x_i \uparrow \Rightarrow \ f(x) \uparrow$$

**Two necessary constraints**:

1. **Non-negative weights**: $W^{(l)} \geq 0$ for all layers (biases unconstrained)
2. **Monotonic activations**: $\sigma'(\cdot) \geq 0$ (ReLU, Sigmoid, Tanh, Softplus)

**Enforcement**: Same two methods as ICNN

- Clamping: simple but can kill neurons
- Reparameterization (Softplus): preferred



# Worked Example: Recovering Preferences via ICNN

**Economic problem: Expenditure minimization**

$$E(p, u) = \min_{c \geq 0} \; p \cdot c$$
$$\text{s.t.} \quad U_\theta(c) \geq u \quad \text{(target utility)}$$

**The "structured" solution**:

- Learn the expenditure function $E_\theta(p, u)$ using a Concave ICNN
- **Theory**: $E(p, u)$ must be concave in prices $p$ (Shephard's Lemma)

**Why this is powerful**:

1. **Shephard's Lemma + Auto-Diff**: $c^*(p, u) = \nabla_p E_\theta(p, u)$ — demand functions for free!
2. **Integrability**: Because $E_\theta$ is structurally concave, the resulting demands satisfy the Slutsky matrix properties by construction

# Summary: Choosing the Right Architecture

Don't use a black box when you have a blueprint.

| Economic Property | Architecture | Key Mechanism |
|---|---|---|
| **Convexity** (Utility, costs) | ICNN | Non-neg weights + skip conn. |
| **Partial Convexity** (Value functions) | Partial ICNN (PICNN) | Split paths (convex/free) |
| **Monotonicity** (Supply/demand) | Monotonic NN<br>Deep Lattice Nets | Non-neg weights<br>Interpolation |
| **Equilibrium** (Market clearing) | Deep Equilibrium (DEQ) | Fixed point iteration |

**Takeaway**: Imposing structure improves interpretability, sample efficiency, and ensures your model respects economic theory

# Accuracy, Validation, and Lab Preview

# Validation Philosophy: The Research Architect Approach

- In computational economics, getting code to *run* is not enough
- The critical question: **Is the answer correct?**
- **Validation strategy** (used in Labs 6A and 6B):
  1. Solve a model with a **known analytical solution** (log utility, Cobb-Douglas)
  2. Implement the numerical method
  3. Compare numerical output against the closed-form truth
  4. Report maximum absolute error across the state space
- **Key metrics**:
  - Policy error: $\max_k |g_{\text{approx}}(k) - g^*(k)|$
  - Value error: $\max_k |V_{\text{approx}}(k) - V^*(k)|$
  - Euler equation error: $\max_k |\mathcal{R}(k; g_{\text{approx}})|$

# Euler Equation Error as a Diagnostic

- Even when no analytical solution exists, we can measure accuracy via the **Euler equation error**:

$$EE(k) = \frac{u'(c)}{\beta \, \mathbb{E}[u'(c') \, f'(k')]} - 1$$

- **Interpretation**: $EE(k) = 0.01$ means the agent is making a 1% optimization error at state $k$
- **Standard benchmarks** (Judd 1998):
  - $\log_{10} |EE| < -3$: acceptable (error $< 0.1\%$)
  - $\log_{10} |EE| < -5$: good (error $< 0.001\%$)
  - $\log_{10} |EE| < -7$: excellent
- This diagnostic works for *any* model with Euler equations, not just those with analytical solutions

# What Lab 6A Establishes: The Ground Truth

**Lab 6A: Classical Benchmark (VFI)**

- Solves the stochastic optimal growth model using VFI with interpolation
- Validates against the analytical solution ($c^* = (1 - \alpha\beta)y$)
- Establishes the "answer key" for Lab 6B

**Key takeaways from Lab 6A**:

1. The benchmark exists: we know *exactly* what the optimal policy looks like
2. VFI works, but requires a grid — breaks in high dimensions
3. The 5-step Research Architect workflow:
   - 3.1 Mathematical formulation (Bellman equation)
   - 3.2 Algorithmic specification (VFI with interpolation)
   - 3.3 Define deliverables (class structure, plots, errors)
   - 3.4 AI-augmented implementation (structured prompting)
   - 3.5 Critical validation (numerical vs. analytical)

# What Lab 6B Demonstrates: Neural Network Solutions

**Lab 6B: Two Deep Learning Approaches**

**Part 1: Euler Equation Minimization**

- Train a `PolicyNetwork` with constrained output to minimize Euler residual
- **Pros**: Very precise policy; theoretically grounded
- **Cons**: No value function; requires deriving FOCs
- Validated against analytical policy $k^* = \alpha\delta k^\alpha$

**Part 2: Deep Value Function Iteration**

- Two networks (`PolicyNet` + `ValueNet`) with $T$-step lookahead and soft constraints
- **Pros**: Provides both policy and value; handles inequality constraints naturally
- **Cons**: Slower convergence; more hyperparameters
- Validated: savings rate $k'/y$ converges to $\alpha\beta$

# Lab Preview: What You Will Do

**Lab 6A: The Classical Benchmark** (VFI)

- Build an `OptimalGrowthModel` class in Python
- Implement VFI with `scipy.interpolate` and `scipy.optimize`
- Validate against the analytical solution
- **Notebook**: `Lab6A_Dynamic_Models.ipynb`

**Lab 6B: Neural Network Solutions** (PyTorch)

- Part 1: Train an Euler equation solver — compare against Lab 6A's ground truth
- Part 2: Train a DVFI solver with $T$-step lookahead and soft constraints
- Compare convergence speed, accuracy, and which method yields richer output
- **Notebook**: `Lab6B_Dynamic_Models.ipynb`

**Goal**: Experience the full pipeline from classical methods to deep learning, using the same model throughout for direct comparison

*For a full treatment of classical VFI, interpolation, and perturbation methods, see the video series at*

## Lecture Summary

1. **The problem**: Grid-based methods break due to the curse of dimensionality
2. **The opportunity**: Neural networks are universal approximators that scale polynomially, not exponentially, with dimension
3. **Two approaches**:
   - **Actor-Critic / DVFI**: Two networks approximate $V(k)$ and $g(k)$; RL flavor
   - **Euler Equation**: One network minimizes the Euler residual; supervised learning flavor
4. **Structured networks** (ICNN, MNN) embed economic theory into the architecture, ensuring well-behaved solutions
5. **Validation**: Always compare against known analytical solutions or Euler equation errors
6. **Next lecture**: Reinforcement learning in depth — the general framework behind actor-critic